

ANTEPROYECTO

IES REY FERNANDO VI



PROgrammer

DONDE EL TALENTO SE DEMUESTRA A TRAVÉS DEL CÓDIGO,
NO DEL TEXTO

COMPONENTES DEL EQUIPO:

Sergio Ajo;
Cristian Escobar;
David Alcázar;

TITULACIÓN Y MODALIDAD:

Ciclo Formativo de Grado Superior en Desarrollo de
Aplicaciones Web (Presencial)

TRABAJO DE FIN DE GRADO

1. INTRODUCCIÓN.....	2
2. ALCANCE DEL PROYECTO Y ANÁLISIS PREVIO.....	4
3. ESTUDIO DE VIABILIDAD.....	5
3.1 ESTADO ACTUAL DEL SISTEMA.....	5
3.2 RESUMEN DE REQUISITOS DEL CLIENTE.....	6
3.3 POSIBLES SOLUCIONES.....	7
3.4 SOLUCIÓN ELEGIDA.....	8
3.5 PLANIFICACIÓN TEMPORAL DEL PROYECTO.....	8
3.6 PLANIFICACIÓN DE LOS RECURSOS A UTILIZAR.....	9
4. ANÁLISIS.....	11
4.1 REQUISITOS FUNCIONALES.....	12
4.2 REQUISITOS NO FUNCIONALES.....	15
5. DISEÑO.....	18
5.1 ESTRUCTURA DE LA APLICACIÓN.....	18
5.2 COMPONENTES DEL SISTEMA Y ARQUITECTURA.....	19
5.3 DISEÑO DE LA BASE DE DATOS.....	20
5.3.1 INTRODUCCIÓN Y JUSTIFICACIÓN.....	20
5.3.2 HERRAMIENTAS UTILIZADAS.....	20
5.3.3 ANÁLISIS DEL MODELO DE DATOS.....	20
5.3.3.1 RELACIONES PRINCIPALES ENTRE TABLAS.....	20
5.3.3.2 DIAGRAMA ENTIDAD-RELACIÓN.....	21
5.3.4 DICCIONARIO DE DATOS DETALLADO.....	22
5.3.5 PROBLEMAS ENCONTRADOS Y SOLUCIONES.....	24
5.3.5.1 DIFERENCIACIÓN Y ESPECIALIZACIÓN DE PERFILES.....	24
5.3.5.2 GESTIÓN DE SELECCIÓN MÚLTIPLE DE TECNOLOGÍAS.....	24
5.3.6 IMPLEMENTACIÓN Y SEGURIDAD.....	25
5.4 HERRAMIENTAS Y TECNOLOGÍAS UTILIZADAS.....	26

1. INTRODUCCIÓN

El presente proyecto, denominado PROgrammer, se desarrolla como trabajo final del ciclo formativo de Desarrollo de Aplicaciones Web (DAW) y tiene como objetivo el diseño e implementación de una plataforma web orientada al ámbito profesional del desarrollo de software. Este proyecto surge a partir de la necesidad de ofrecer una alternativa a las redes profesionales tradicionales, en las que la evaluación de los candidatos se basa principalmente en currículums y descripciones teóricas, lo que no siempre refleja de manera precisa sus capacidades reales.

Frente a esta problemática, PROgrammer plantea un enfoque centrado en la demostración práctica de habilidades, permitiendo a los desarrolladores mostrar sus conocimientos a través de proyectos reales desarrollados por ellos mismos. Estos proyectos forman parte del perfil del usuario dentro de la plataforma y pueden ser evaluados mediante un sistema de valoraciones y revisiones técnicas, lo que permite analizar aspectos como la calidad del desarrollo, la organización del proyecto o la documentación aportada. De este modo, se busca establecer un entorno más transparente y fiable tanto para los desarrolladores como para las empresas.

El desarrollo del proyecto ha sido llevado a cabo por Sergio Ajo, Cristian Escobar y David Alcázar, y donde se han aplicado de forma conjunta los conocimientos adquiridos a lo largo del ciclo formativo en los distintos módulos profesionales. Para ello, se ha diseñado una arquitectura web completa que integra tanto tecnologías de entorno cliente como de entorno servidor. En la parte frontend se han utilizado HTML, CSS, JavaScript, jQuery y Bootstrap, con el objetivo de construir una interfaz moderna, intuitiva y adaptable a distintos dispositivos. Por otro lado, el backend se desarrollará mediante el framework Spring Boot, permitiendo gestionar la lógica de negocio y la comunicación con la base de datos, la cual se ha implementado utilizando MySQL.

Además, la plataforma incorpora diferentes funcionalidades orientadas a mejorar la experiencia del usuario, como sistemas de búsqueda diferenciados para desarrolladores y empresas, herramientas de exploración de perfiles y proyectos, así como mecanismos de valoración y validación de contenido, lo que permite recrear un entorno profesional enfocado específicamente al sector del desarrollo de software.

A lo largo de esta memoria se describirán en detalle los distintos aspectos que han formado parte del desarrollo del proyecto, incluyendo el análisis de requisitos, el diseño del sistema, la

planificación, la implementación y las pruebas realizadas. De esta forma, se pretende ofrecer una visión completa tanto del resultado final como del proceso seguido para su desarrollo.

2. ALCANCE DEL PROYECTO Y ANÁLISIS PREVIO

El presente proyecto, denominado PROgrammer, tiene como propósito el desarrollo de una plataforma web orientada al ámbito profesional del desarrollo de software, cuyo objetivo principal es ofrecer un entorno en el que los desarrolladores puedan demostrar sus habilidades mediante proyectos reales, facilitando al mismo tiempo su visibilidad frente a empresas interesadas en la captación de talento técnico.

A diferencia de otros sistemas basados principalmente en la descripción teórica de conocimientos, este proyecto plantea un enfoque centrado en la práctica, permitiendo que los usuarios puedan exponer de forma directa los resultados de su trabajo a través de proyectos desarrollados por ellos mismos. De este modo, se pretende mejorar los procesos de evaluación dentro del sector, proporcionando una visión más objetiva y completa de las capacidades de cada desarrollador.

Asimismo, la plataforma se concibe como un punto de encuentro entre desarrolladores y empresas, permitiendo a estas últimas buscar perfiles técnicos, consultar proyectos y publicar ofertas de empleo, mientras que los desarrolladores pueden gestionar su perfil, mostrar sus trabajos y postularse a dichas ofertas. Todo ello se integra en un entorno único que centraliza la información relevante, facilitando su acceso y gestión.

En este sentido, el objetivo general del proyecto consiste en diseñar e implementar una aplicación web funcional que permita la gestión de usuarios, perfiles técnicos, proyectos y valoraciones, así como la interacción entre desarrolladores y empresas dentro de un entorno orientado específicamente al sector del desarrollo de software.

El desarrollo de este proyecto se ha estructurado en varias fases, las cuales permiten organizar de manera progresiva las distintas tareas necesarias para su realización. En primer lugar, se ha llevado a cabo un estudio de viabilidad, en el que se ha analizado la situación actual y se han valorado las posibles soluciones. Posteriormente, se ha realizado la fase de análisis, en la que se han definido los requisitos que debe cumplir el sistema.

A continuación, se abordará la fase de diseño, en la que se establecerá la arquitectura del sistema y la estructura de sus distintos componentes. Seguidamente, se llevará a cabo la fase de implementación, en la que se desarrollarán las funcionalidades definidas previamente. Tras ello, se realizará la fase de pruebas, con el objetivo de verificar el correcto funcionamiento del sistema y detectar posibles errores. Finalmente, se contempla una fase de explotación o ejecución, en la que la aplicación podrá ser utilizada en un entorno real.

A lo largo de los siguientes apartados se describirá en detalle cada una de estas fases, así como las actividades y procedimientos asociados a las mismas, permitiendo obtener una visión completa del proceso de desarrollo del proyecto.

3. ESTUDIO DE VIABILIDAD

En la presente fase se lleva a cabo un análisis inicial con el objetivo de determinar la viabilidad del proyecto PROgrammer, teniendo en cuenta tanto los recursos disponibles como las posibles limitaciones técnicas y funcionales. Este estudio permite evaluar si el desarrollo de la aplicación resulta factible dentro del contexto académico en el que se enmarca, así como identificar las diferentes alternativas existentes y seleccionar la solución más adecuada en función de los objetivos planteados.

Para ello, se realiza en primer lugar un análisis del estado actual del sistema, con el fin de comprender la situación de partida y las carencias existentes en el ámbito al que se dirige el proyecto. Posteriormente, se establece un resumen de los requisitos que se pretenden satisfacer, definiendo de manera clara las necesidades que motivan el desarrollo de la plataforma. Finalmente, se valoran distintas soluciones posibles, justificando la elección de la alternativa adoptada.

3.1 ESTADO ACTUAL DEL SISTEMA

En la actualidad, los procesos de búsqueda de empleo y evaluación de candidatos dentro del sector del desarrollo de software se apoyan principalmente en plataformas profesionales generalistas y en el uso de currículums tradicionales. Estas herramientas, si bien resultan útiles para ofrecer una visión general del perfil de un candidato, presentan ciertas limitaciones a la hora de reflejar de manera objetiva sus capacidades técnicas reales.

En este contexto, los desarrolladores suelen recurrir a repositorios externos para alojar sus proyectos, los cuales deben ser compartidos de forma manual durante los procesos de selección. Esto implica que la información relevante se encuentra dispersa en distintas plataformas, dificultando tanto su acceso como su evaluación por parte de las empresas.

Por otro lado, los sistemas actuales no suelen incorporar mecanismos estructurados que permitan valorar de forma homogénea aspectos técnicos como la calidad del desarrollo, la organización del proyecto o la documentación aportada. Esta situación provoca que la valoración de los candidatos dependa en gran medida de interpretaciones subjetivas, lo que puede derivar en procesos de selección menos precisos.

En consecuencia, se identifica la necesidad de una solución que centralice la información técnica de los desarrolladores y permita su evaluación de forma más clara, estructurada y accesible.

3.2 RESUMEN DE REQUISITOS DEL CLIENTE

El proyecto PROgrammer surge con el objetivo de dar respuesta a la necesidad de disponer de una plataforma especializada que permita a los desarrolladores mostrar sus habilidades a través de proyectos reales, facilitando al mismo tiempo su visibilidad frente a posibles empleadores.

En este sentido, se establecen como requisitos principales la posibilidad de que los usuarios puedan registrarse en la plataforma y gestionar su perfil personal, así como crear un perfil de desarrollador en el que se recojan sus proyectos, tecnologías dominadas y valoraciones recibidas.

Asimismo, se requiere la implementación de un sistema que permita la subida de proyectos, incluyendo información relevante como la descripción, las tecnologías utilizadas, material visual y la posibilidad de descarga. Estos proyectos deberán poder ser valorados mediante un sistema de puntuación basado en estrellas y comentarios, permitiendo diferenciar aquellos que han sido revisados mediante una validación específica.

Por otro lado, la plataforma deberá ofrecer funcionalidades orientadas a las empresas, permitiendo la búsqueda de desarrolladores y la publicación de ofertas de empleo, así como la consulta de perfiles técnicos basados en proyectos reales.

Finalmente, se contempla la necesidad de incorporar sistemas de búsqueda que faciliten la localización de usuarios y contenido dentro de la plataforma, así como otros elementos orientados a mejorar la experiencia de usuario. **¡AQUÍ SE HABLARÁ DE FUNCIONALIDADES ADICIONALES COMO LA MENSAJERÍA!**

3.3 POSIBLES SOLUCIONES

Una vez analizada la situación actual y definidos los requisitos del proyecto, se procede a estudiar diferentes alternativas que podrían dar respuesta a las necesidades detectadas. Estas soluciones se valoran en función de su viabilidad técnica, su grado de adaptación al problema planteado y su capacidad para ofrecer un valor diferencial respecto a las herramientas existentes.

En primer lugar, se podría considerar el uso de plataformas profesionales generalistas ya existentes, las cuales permiten a los usuarios crear perfiles, incluir información sobre su experiencia y establecer contacto con empresas. Sin embargo, estas herramientas no están específicamente diseñadas para el ámbito del desarrollo de software, lo que limita su capacidad para reflejar de forma precisa las habilidades técnicas de los usuarios. Además, el uso predominante del currículum como elemento central dificulta la evaluación práctica del talento.

Por otro lado, existen plataformas orientadas al alojamiento de proyectos de desarrollo, en las que los usuarios pueden compartir su código y colaborar con otros desarrolladores. Si bien estas herramientas resultan útiles desde el punto de vista técnico, no están concebidas como entornos de selección profesional, por lo que carecen de funcionalidades específicas para la evaluación estructurada de proyectos o la conexión directa con empresas.

Otra posible solución consistiría en combinar el uso de diferentes herramientas, utilizando plataformas profesionales para la creación de perfiles y repositorios externos para la gestión de proyectos. No obstante, esta alternativa implica una fragmentación de la información, lo que dificulta tanto la gestión por parte del usuario como su análisis por parte de las empresas.

Finalmente, se plantea como alternativa el desarrollo de una plataforma propia que integre en un único entorno todas las funcionalidades necesarias, permitiendo centralizar la información y adaptar el sistema de forma específica a las necesidades del sector del desarrollo de software.

3.4 SOLUCIÓN ELEGIDA

Tras analizar las diferentes alternativas, se ha optado por el desarrollo de una solución propia, materializada en la plataforma PROgrammer, al considerarse la opción más adecuada para satisfacer los requisitos planteados y ofrecer un valor diferencial respecto a las herramientas existentes.

Esta decisión se fundamenta en la posibilidad de diseñar un sistema completamente adaptado al objetivo del proyecto, permitiendo que los desarrolladores puedan mostrar sus capacidades a través de proyectos reales, los cuales pueden ser valorados y consultados de manera centralizada. De este modo, se elimina la necesidad de recurrir a múltiples plataformas externas, facilitando tanto la gestión de la información como su acceso por parte de las empresas.

Asimismo, el desarrollo de una solución propia permite implementar funcionalidades específicas que no se encuentran presentes de forma conjunta en otras herramientas, como la integración de sistemas de valoración estructurados, la diferenciación entre perfiles de usuario y perfiles técnicos, o la posibilidad de destacar aquellos proyectos que han sido revisados mediante procesos de validación.

Por otro lado, esta alternativa ofrece una mayor flexibilidad a la hora de ampliar o modificar el sistema en el futuro, permitiendo incorporar nuevas funcionalidades en función de las necesidades detectadas durante su uso.

En consecuencia, se concluye que el desarrollo de PROgrammer constituye la solución más adecuada, al permitir abordar de forma directa las limitaciones identificadas en el estado actual del sistema y ofrecer una propuesta orientada específicamente al ámbito del desarrollo de software.

3.5 PLANIFICACIÓN TEMPORAL DEL PROYECTO

[PLACEHOLDER – AQUÍ IRÁ LA PLANIFICACIÓN TEMPORAL DE PROYECTO (EL PUNTO 3.5)]

La planificación temporal detallada del proyecto, incluyendo la distribución de tareas y su estimación en el tiempo, se desarrollará en fases posteriores de la memoria, donde se reflejará

de manera más precisa el desarrollo real del proyecto mediante herramientas como el diagrama de Gantt.

3.6 PLANIFICACIÓN DE LOS RECURSOS A UTILIZAR

Para el desarrollo del proyecto PROgrammer ha sido necesario disponer de un conjunto de recursos tanto humanos como materiales y tecnológicos, los cuales han permitido abordar las distintas fases del proyecto de forma organizada, garantizando así la viabilidad del mismo dentro del contexto académico en el que se enmarca. La correcta planificación de estos recursos resulta fundamental, ya que permite optimizar el tiempo disponible y asegurar una distribución equilibrada de las tareas entre los miembros del equipo.

En lo que respecta a los recursos humanos, el proyecto ha sido desarrollado por un equipo compuesto por tres integrantes, quienes han trabajado de manera coordinada desde sus respectivos entornos personales. Cada uno de los miembros ha participado activamente en el desarrollo del sistema, asumiendo diferentes responsabilidades en función de las necesidades del proyecto en cada momento. Esta organización ha permitido abordar de manera simultánea distintas tareas, favoreciendo el avance progresivo del desarrollo y reduciendo posibles cuellos de botella. Asimismo, la toma de decisiones se ha realizado de forma conjunta, fomentando un entorno colaborativo en el que se han valorado las distintas propuestas antes de su implementación.

En cuanto a los recursos materiales, se ha contado con equipos informáticos personales adecuados para el desarrollo de aplicaciones web, así como con acceso continuo a conexión a internet. Este último elemento ha resultado especialmente relevante, no solo para el desarrollo técnico del proyecto, sino también para la utilización de herramientas colaborativas que han permitido mantener una comunicación constante entre los integrantes del equipo, así como el acceso a repositorios remotos y servicios de almacenamiento en la nube.

Por otro lado, en relación con los recursos software, se han empleado diversas herramientas que han facilitado tanto el desarrollo como la organización del trabajo. Para la implementación del frontend se ha utilizado el entorno de desarrollo Visual Studio Code, el cual ha permitido trabajar de forma eficiente con tecnologías como HTML, CSS, JavaScript, Bootstrap, jQuery y jQuery UI, facilitando la creación de interfaces dinámicas y adaptables a distintos dispositivos.

Asimismo, se ha utilizado GitHub como sistema de control de versiones y repositorio remoto, lo que ha permitido gestionar el desarrollo simultáneo del proyecto. Mediante el uso de

operaciones como commits, pushes y pulls, se ha podido mantener un control detallado de los cambios realizados, así como garantizar la integridad del código en todo momento. Esta herramienta ha resultado clave para coordinar el trabajo del equipo, evitando conflictos y permitiendo la integración progresiva de las distintas funcionalidades desarrolladas.

De forma complementaria, se ha hecho uso de Google Drive como plataforma de almacenamiento en la nube, en la que se han gestionado documentos, recursos gráficos, material de apoyo y otros elementos necesarios para el desarrollo del proyecto que no forman parte directa del código. Asimismo, se ha utilizado Discord como herramienta principal de comunicación, permitiendo la realización de reuniones en tiempo real, así como el intercambio rápido de información entre los miembros del equipo.

En lo relativo a la gestión de datos, se ha empleado MySQL como sistema de base de datos, gestionado a través de herramientas como phpMyAdmin, lo que ha permitido diseñar y estructurar la persistencia de la información necesaria para el funcionamiento de la plataforma.

[AQUÍ SE HABLARÁ DEL BACKEND]

En cuanto a las herramientas de apoyo al diseño, se han utilizado aplicaciones como Pencil para la elaboración de prototipos de interfaz, lo que ha facilitado la planificación previa de la estructura visual de la aplicación antes de su implementación. Del mismo modo, se ha utilizado Adobe Photoshop para la creación y edición de elementos gráficos, tales como logotipos e imágenes representativas, contribuyendo a dotar al proyecto de una identidad visual coherente.

Respecto a los conocimientos necesarios para el desarrollo del proyecto, se han aplicado tanto los adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Web como aquellos obtenidos de forma autodidacta por parte de los integrantes del equipo. Este aprendizaje adicional ha resultado especialmente relevante en aspectos como el uso de sistemas de control de versiones en entornos colaborativos o la profundización en determinadas tecnologías necesarias para el desarrollo del proyecto. En este sentido, se ha realizado un esfuerzo adicional en la comprensión de herramientas y frameworks relacionados con el desarrollo backend, con el objetivo de preparar la futura implementación del sistema. **[AQUÍ SE HABLARÁ**

DE TECNOLOGÍAS ADICIONALES COMO REACT]

Finalmente, en lo referente a la metodología de trabajo, se ha adoptado un enfoque colaborativo basado en la coordinación continua entre los miembros del equipo. Siempre que ha sido posible, se han llevado a cabo sesiones de trabajo conjuntas mediante herramientas de comunicación en línea, en las que cada integrante ha desarrollado tareas específicas de forma

simultánea, evitando interferencias en el trabajo de los demás. Este sistema ha permitido avanzar de manera eficiente en distintas partes del proyecto de forma paralela.

En aquellos casos en los que no ha sido posible trabajar de forma simultánea, se han establecido reuniones previas destinadas a la planificación de las tareas a realizar, definiendo claramente las responsabilidades de cada integrante. Posteriormente, se han realizado revisiones conjuntas o verificaciones individuales del trabajo realizado, con el objetivo de asegurar la coherencia del proyecto y detectar posibles errores o inconsistencias.

Asimismo, los cambios relevantes en el repositorio han sido sometidos a revisiones individuales por parte de los miembros del equipo antes de su integración definitiva, garantizando de este modo la calidad del código y el correcto funcionamiento del sistema. Este proceso ha permitido mantener un control riguroso sobre la evolución del proyecto, minimizando riesgos y asegurando un desarrollo progresivo y estructurado.

En base a todo lo anterior, se concluye que el proyecto PROgrammer resulta viable tanto desde el punto de vista técnico como organizativo, permitiendo abordar su desarrollo dentro del contexto establecido y sentando las bases para las siguientes fases del proyecto.

4. ANÁLISIS

En la presente fase se lleva a cabo la definición detallada de los requisitos que deberá cumplir el sistema PROgrammer, una vez aceptada la propuesta del proyecto y establecido su alcance general. Este proceso resulta fundamental, ya que permite concretar de manera precisa las funcionalidades y características que deberá incorporar la plataforma, sirviendo como base para las fases posteriores de diseño e implementación.

El análisis del sistema tiene como objetivo principal identificar las necesidades que se pretenden cubrir mediante el desarrollo de la aplicación, así como establecer las condiciones bajo las cuales deberá operar. Para ello, se realiza una especificación estructurada de los requisitos, diferenciando entre aquellos de carácter funcional, que describen las acciones que el sistema debe ser capaz de realizar, y aquellos de carácter no funcional, que definen las propiedades y cualidades que debe cumplir el sistema en su conjunto.

En este sentido, el sistema PROgrammer se concibe como una plataforma web orientada al ámbito profesional del desarrollo de software, cuyo propósito es permitir a los usuarios mostrar

sus habilidades mediante proyectos reales, así como facilitar la interacción entre desarrolladores y empresas. A partir de este planteamiento, se establecen una serie de funcionalidades clave que deberán ser implementadas, entre las que destacan la gestión de usuarios, la administración de perfiles técnicos, la publicación y valoración de proyectos, así como la posibilidad de consultar ofertas de empleo y establecer contacto con entidades interesadas.

Asimismo, el sistema deberá contemplar la existencia de distintos tipos de usuario, cada uno con permisos y funcionalidades específicas. Entre ellos se encuentran los desarrolladores, que podrán gestionar sus perfiles y proyectos; las empresas, que podrán consultar perfiles y publicar ofertas de empleo; y los administradores o evaluadores, que dispondrán de capacidades adicionales relacionadas con la validación de contenido dentro de la plataforma.

Por otro lado, se tendrá en cuenta la necesidad de incorporar mecanismos que permitan garantizar una experiencia de usuario adecuada, así como facilitar la navegación y el acceso a la información dentro del sistema. En este sentido, se contemplan funcionalidades como sistemas de búsqueda diferenciados en función del tipo de usuario, así como herramientas orientadas a la interacción entre los distintos perfiles. **AQUÍ SE HABLARÁ DE FUNCIONALIDADES ADICIONALES COMO LA MENSAJERÍA**

A lo largo de los siguientes apartados se detallarán de forma específica los distintos requisitos del sistema, así como los modelos y representaciones necesarias para comprender su funcionamiento, incluyendo los requisitos funcionales y no funcionales, los diagramas de casos de uso y el modelo de datos que servirá de base para la implementación del sistema.

4.1 REQUISITOS FUNCIONALES

Los requisitos funcionales describen de manera detallada las funcionalidades que el sistema debe proporcionar, es decir, las acciones y comportamientos que la plataforma PROgrammer deberá ser capaz de ejecutar para satisfacer las necesidades de los usuarios. Estos requisitos permiten definir el alcance real del sistema y sirven como base para su posterior diseño e implementación.

En el caso del proyecto PROgrammer, los requisitos funcionales se han definido teniendo en cuenta los distintos tipos de usuario que interactúan con la plataforma, así como las funcionalidades específicas asociadas a cada uno de ellos. A continuación, se presentan los principales requisitos funcionales identificados:

Relativo al registro y gestión de usuarios:

- RF1. Registro de usuarios:
El sistema deberá permitir a nuevos usuarios registrarse en la plataforma mediante un formulario en el que se recogerán los datos necesarios para la creación de una cuenta.
- RF2. Inicio de sesión:
El sistema deberá permitir a los usuarios autenticarse mediante sus credenciales para acceder a las funcionalidades disponibles según su rol.
- RF3. Gestión de sesión:
El sistema deberá gestionar la sesión del usuario una vez autenticado, permitiendo mantener su estado dentro de la plataforma y finalizando dicha sesión cuando el usuario así lo indique.
- RF4. Creación de perfil de desarrollador:
El sistema deberá permitir a los usuarios crear un perfil técnico de desarrollador independiente de su perfil personal, en el que se recoja información relevante sobre sus habilidades.

Relativo a la vista de perfil de desarrollador:

- RF5. Visualización de perfil de desarrollador:
El sistema deberá permitir consultar los perfiles de desarrollador, incluyendo información como proyectos, tecnologías dominadas, valoraciones y datos adicionales.
- RF6. Gestión de información del perfil:
El usuario deberá poder editar la información de su perfil, incluyendo descripción, tecnologías y otros datos relevantes.
- RF7. Descarga de currículum:
El sistema deberá permitir a los desarrolladores adjuntar y poner a disposición su currículum para su descarga por parte de otros usuarios o empresas.

Relativo a la vista de proyectos:

- RF8. Publicación de proyectos:
El sistema deberá permitir a los desarrolladores subir proyectos a la plataforma, incluyendo información como nombre, descripción, tecnologías utilizadas, imágenes representativas y un archivo descargable.

- RF9. Visualización de proyectos:
El sistema deberá permitir consultar los proyectos publicados, mostrando la información asociada a cada uno de ellos.
- RF10. Descarga de proyectos:
El sistema deberá permitir a los usuarios descargar los archivos asociados a los proyectos publicados.
- RF11. Identificación del estado del proyecto:
El sistema deberá mostrar si un proyecto ha sido revisado mediante un distintivo visual que indique su validación por parte de un evaluador o administrador.
- RF12. Valoración de proyectos:
El sistema deberá permitir a los usuarios valorar proyectos mediante un sistema basado en puntuaciones (1 a 5 estrellas) y comentarios.
- RF13. Visualización de valoraciones de proyectos:
El sistema deberá permitir consultar las valoraciones asociadas a cada proyecto.

Relativo a las reseñas y valoraciones:

- RF14. Valoración de perfiles:
El sistema deberá permitir a los usuarios valorar perfiles de desarrollador mediante comentarios y puntuaciones.
- RF15. Validación por evaluador:
El sistema deberá permitir a los administradores o evaluadores revisar proyectos y otorgarles un estado de validación que los diferencie del resto.

Relativo a las funciones de aplicación para el perfil de empresa:

- RF16. Búsqueda de desarrolladores:
El sistema deberá permitir a las empresas buscar perfiles de desarrollador mediante un sistema de búsqueda.
- RF17. Publicación de ofertas de empleo:
El sistema deberá permitir a las empresas publicar ofertas de empleo dentro de la plataforma.

Relativo a las funciones de aplicación para el perfil de desarrollador:

- RF18. Visualización de ofertas:
El sistema deberá permitir a los usuarios consultar las ofertas de empleo disponibles.

- RF19. Postulación a ofertas:

El sistema deberá permitir a los desarrolladores postularse a ofertas de empleo publicadas por las empresas.

Relativo a las funciones de búsqueda.

- RF20. Búsqueda de usuarios:

El sistema deberá permitir la búsqueda de usuarios en función de distintos criterios básicos, como el nombre.

- RF21. Búsqueda de empresas:

El sistema deberá permitir la búsqueda de empresas registradas dentro de la plataforma.

[AQUÍ SE HABLARÁ DE MEJORAS FUTURAS EN EL SISTEMA DE BÚSQUEDA]

Relativo a la interacción entre perfiles:

- RF22. Interacción entre usuarios:

El sistema podrá incorporar funcionalidades orientadas a la comunicación entre usuarios, con el objetivo de facilitar la interacción dentro de la plataforma.

[AQUÍ SE HABLARÁ DE LA MENSAJERÍA]

En conjunto, estos requisitos funcionales definen el comportamiento esperado del sistema PROgrammer, estableciendo las bases sobre las cuales se desarrollará la aplicación. Su correcta identificación permite garantizar que el sistema final responda a las necesidades planteadas inicialmente, asegurando así la coherencia entre los objetivos del proyecto y su implementación.

4.2 REQUISITOS NO FUNCIONALES

Los requisitos no funcionales definen las propiedades y características que debe cumplir el sistema PROgrammer en su conjunto, sin describir acciones específicas, sino estableciendo criterios de calidad, rendimiento y usabilidad. Estos requisitos resultan fundamentales, ya que determinan cómo debe comportarse el sistema desde un punto de vista global, influyendo directamente en la experiencia del usuario y en la fiabilidad de la aplicación.

A diferencia de los requisitos funcionales, que se centran en las tareas que el sistema debe realizar, los requisitos no funcionales establecen las condiciones bajo las cuales dichas tareas

deben ejecutarse, garantizando que el sistema sea usable, eficiente y adecuado para el contexto en el que se desarrolla.

A continuación, se detallan los principales requisitos no funcionales identificados para el proyecto:

Relativos a la usabilidad:

- RNF1. Interfaz intuitiva:
El sistema deberá presentar una interfaz clara, organizada y de fácil comprensión, permitiendo a los usuarios navegar por la plataforma sin necesidad de conocimientos técnicos avanzados.
- RNF2. Diseño adaptable (responsive):
La aplicación deberá adaptarse correctamente a distintos dispositivos y tamaños de pantalla, garantizando una experiencia de usuario adecuada tanto en ordenadores como en dispositivos móviles.
- RNF3. Facilidad de navegación:
El sistema deberá estructurar la información de forma lógica, facilitando el acceso a las distintas secciones de la plataforma.

Relativos al rendimiento:

- RNF4. Tiempo de respuesta:
El sistema deberá ofrecer tiempos de carga adecuados en las distintas operaciones, evitando demoras que puedan afectar negativamente a la experiencia del usuario.
- RNF5. Gestión eficiente de recursos:
La aplicación deberá optimizar el uso de los recursos disponibles, garantizando un funcionamiento fluido incluso en condiciones de uso moderado.

Relativos a la seguridad:

- RNF6. Protección de datos de usuario:
El sistema deberá garantizar la protección de los datos personales de los usuarios, evitando accesos no autorizados.
- RNF7. Control de acceso:
El sistema deberá gestionar correctamente los permisos en función del tipo de usuario, asegurando que cada uno acceda únicamente a las funcionalidades que le correspondan.

- RNF8. Validación de datos:

La aplicación deberá incorporar mecanismos de validación de datos en los formularios, con el objetivo de evitar errores y posibles vulnerabilidades.

Relativos a la mantenibilidad y escalabilidad:

- RNF9. Estructura modular:

El sistema deberá estar organizado de forma estructurada, facilitando su mantenimiento y futuras ampliaciones.

- RNF10. Posibilidad de ampliación:

La aplicación deberá permitir la incorporación de nuevas funcionalidades en el futuro sin necesidad de realizar modificaciones estructurales complejas.

- Relativos a la fiabilidad:

- RNF11. Consistencia de la información:

El sistema deberá garantizar que los datos almacenados se mantengan coherentes y actualizados en todo momento.

- RNF12. Disponibilidad del sistema:

La plataforma deberá estar disponible para su uso en condiciones normales, evitando interrupciones en la medida de lo posible.

Además de los requisitos anteriormente descritos, el desarrollo del proyecto se encuentra sujeto a una serie de restricciones que condicionan su alcance y ejecución.

En primer lugar, se trata de un proyecto desarrollado en el contexto de un ciclo formativo, lo que implica limitaciones tanto en el tiempo disponible como en los recursos técnicos. Por este motivo, algunas funcionalidades planteadas inicialmente podrán ser simplificadas o pospuestas para futuras versiones del sistema.

Asimismo, determinadas partes del sistema, como la implementación completa del backend o funcionalidades avanzadas como los sistemas de comunicación entre usuarios, se encuentran sujetas a desarrollo en fases posteriores. **[AQUÍ SE HABLARÁ DEL BACKEND Y DE FUNCIONALIDADES FUTURAS]**

Por otro lado, el sistema se basa en tecnologías concretas previamente definidas, lo que condiciona tanto su diseño como su implementación.

En conjunto, los requisitos no funcionales establecen las bases que garantizan la calidad del sistema PROgrammer, definiendo las condiciones necesarias para que la aplicación no solo

cumpla con las funcionalidades previstas, sino que lo haga de forma eficiente, segura y adecuada para los usuarios.

[AQUÍ DEBEMOS AÑADIR EL DIAGRAMA DE CASOS DE USO COMO SUBAPARTADO EXTRA]

[AQUÍ DEBEMOS AÑADIR EL MODELO DE DATOS ENTIDAD-RELACIÓN COMO SUBAPARTADO EXTRA]

5. DISEÑO

En la presente fase se realiza una aproximación al diseño tecnológico del sistema PROgrammer, definiendo la estructura general de la aplicación, los componentes que la integran y las tecnologías necesarias para su desarrollo. Este proceso resulta fundamental, ya que permite establecer una base sólida sobre la que se implementarán posteriormente las funcionalidades definidas en la fase de análisis.

El diseño del sistema se ha planteado teniendo en cuenta los requisitos previamente establecidos, con el objetivo de garantizar una correcta organización de los distintos elementos que componen la aplicación, así como su escalabilidad y mantenibilidad a largo plazo. De este modo, se definen tanto la estructura de la aplicación como la arquitectura general del sistema, permitiendo comprender cómo interactúan entre sí los distintos componentes.

5.1 ESTRUCTURA DE LA APLICACIÓN

El proyecto PROgrammer se define como una aplicación web, accesible a través de un navegador, lo que permite a los usuarios interactuar con el sistema sin necesidad de instalar software adicional en sus dispositivos. Este enfoque facilita la accesibilidad y el uso de la plataforma desde distintos entornos, favoreciendo su adopción por parte de los usuarios.

La aplicación se estructura siguiendo un modelo basado en la separación de responsabilidades, diferenciando claramente entre la capa de presentación (frontend), la lógica de negocio y la gestión de datos. Esta organización permite mantener una arquitectura ordenada y facilita tanto el desarrollo como el mantenimiento del sistema.

En la capa de presentación se encuentra la interfaz de usuario, desarrollada mediante tecnologías web como HTML, CSS y JavaScript, apoyadas por frameworks y librerías que

permiten mejorar la experiencia de usuario y la adaptabilidad de la aplicación. Por otro lado, la lógica de negocio se encarga de procesar las operaciones realizadas por los usuarios y gestionar la comunicación con la base de datos. **[AQUÍ SE HABLARÁ DEL BACKEND]**

Finalmente, la capa de datos se encarga de la persistencia de la información, permitiendo almacenar y recuperar los datos necesarios para el funcionamiento del sistema de forma estructurada y eficiente.

5.2 COMPONENTES DEL SISTEMA Y ARQUITECTURA

El sistema PROgrammer se basa en una arquitectura cliente-servidor, en la que se diferencian claramente los distintos elementos que intervienen en su funcionamiento.

Por un lado, se encuentra el cliente, representado por el navegador web del usuario, desde el cual se accede a la plataforma y se realizan las distintas operaciones disponibles. Este componente es el encargado de mostrar la interfaz de usuario y de enviar las solicitudes al servidor.

Por otro lado, se encuentra el servidor de aplicaciones, responsable de procesar las peticiones recibidas, aplicar la lógica de negocio correspondiente y gestionar la comunicación con la base de datos. **[AQUÍ SE HABLARÁ DEL BACKEND]**

Asimismo, el sistema cuenta con un servidor de base de datos, en el que se almacena toda la información necesaria para el funcionamiento de la plataforma, incluyendo usuarios, perfiles, proyectos, valoraciones y ofertas de empleo.

Esta arquitectura permite separar claramente las responsabilidades de cada componente, facilitando la escalabilidad del sistema y permitiendo futuras ampliaciones sin necesidad de realizar modificaciones estructurales significativas.

[AQUÍ SE INCORPORARÁ UN DIAGRAMA DE ARQUITECTURA DEL SISTEMA]

En lo relativo al despliegue del sistema, se contempla su implementación en un entorno de servidor real, lo que permitirá su acceso a través de internet por parte de los usuarios finales. No obstante, los detalles específicos de esta fase se definirán en etapas posteriores del proyecto. **[AQUÍ SE HABLARÁ DEL DESPLIEGUE DEL SISTEMA]**

5.3 DISEÑO DE LA BASE DE DATOS

5.3.1 INTRODUCCIÓN Y JUSTIFICACIÓN

- Modelo de Persistencia: Se ha optado por un sistema de gestión relacional para facilitar la integración con Spring Data JPA. Esto permite que el esquema de la base de datos sea el reflejo exacto del modelo de dominio del backend, asegurando una persistencia de datos robusta y transaccional.
- Escalabilidad: El diseño permite el crecimiento horizontal de las tablas de contenido (mensajes y proyectos) mediante el uso de claves indexadas, lo que garantiza tiempos de respuesta rápidos incluso con un volumen de usuarios creciente.

5.3.2 HERRAMIENTAS UTILIZADAS

- MySQL 8.0: Como sistema de gestión de base de datos (SGBD) principal por su escalabilidad y compatibilidad con el entorno Spring Boot.
- MySQL Workbench: Herramienta fundamental para el modelado visual del diagrama Entidad-Relación (E-R) y la ingeniería directa hacia el servidor.

5.3.3 ANÁLISIS DEL MODELO DE DATOS

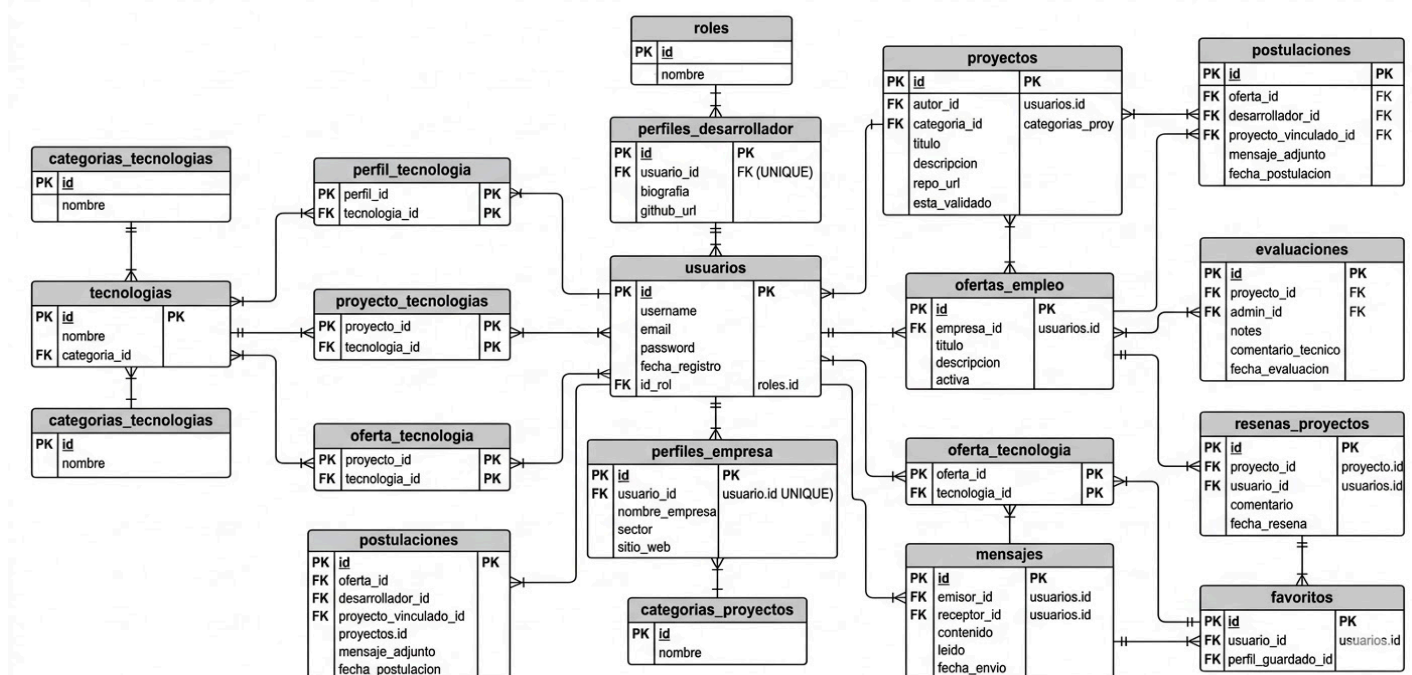
5.3.3.1 RELACIONES PRINCIPALES ENTRE TABLAS

El sistema se articula sobre varios ejes relacionales identificados en el script de creación:

1. Especialización de Usuarios (1:1): La tabla usuarios actúa como entidad base. Se implementa una relación de uno a uno con perfiles_desarrollador y perfiles_empresa, permitiendo que cada usuario tenga atributos específicos según su rol (Admin, Developer, Company, Visitor).
2. Gestión de Tecnologías (N:M): Para permitir que un proyecto o un desarrollador posea múltiples habilidades, se han creado tablas intermedias como perfil_tecnologia, proyecto_tecnologias y oferta_tecnologia.
3. Sistema de Postulaciones (Relación Ternaria): La tabla postulaciones vincula una oferta_id, un desarrollador_id y un proyecto_vinculado_id, permitiendo que el candidato demuestre sus capacidades con un trabajo específico al aplicar a una oferta.

4. Mensajería Autoreferenciada (1:N): La tabla mensajes utiliza claves foráneas (emisor_id y receptor_id) que apuntan ambas a la tabla usuarios, permitiendo la comunicación fluida entre cualquier tipo de perfil.
5. Optimización de la Especialización: Esta estructura de 1:1 mediante claves foráneas únicas (usuario_id) permite una segregación de responsabilidades clara. Evita que la tabla usuarios contenga campos nulos masivos (como URLs de GitHub para empresas), optimizando así el almacenamiento y la velocidad de indexación en el motor MySQL.
6. Integridad en Relaciones N:M: Las tablas intermedias no solo vinculan entidades, sino que definen claves primarias compuestas (por ejemplo, perfil_id y tecnologia_id en perfil_tecnologia) para impedir la duplicidad de registros. Esto garantiza que un desarrollador no pueda marcar la misma tecnología dos veces en su perfil.
7. Lógica de Negocio en la Relación Ternaria: La tabla postulaciones es el núcleo operativo de la plataforma. Al exigir un proyecto_vinculado_id como clave foránea obligatoria, el sistema impone por diseño que cualquier postulación vaya acompañada de una evidencia técnica del portfolio del usuario, reforzando la calidad de los procesos de selección en ProGrammer.
8. Trazabilidad en Mensajería: El uso de una relación autoreferenciada sobre la tabla usuarios permite centralizar la lógica de comunicación. Al gestionar tanto emisores como receptores desde la misma entidad, se facilita la implementación de sistemas de notificaciones dinámicas en el Header de la aplicación.

5.3.3.2 DIAGRAMA ENTIDAD-RELACIÓN



5.3.4 DICCIONARIO DE DATOS DETALLADO

A continuación, se describen las tablas maestras que sostienen la lógica de negocio:

Tabla	Propósito	Claves Principales / Foráneas
usuarios	Registro central de credenciales y acceso al sistema.	id_rol (FK a roles)
perfiles_desarrollador	Almacena biografía, enlaces a GitHub y datos técnicos de los programadores.	usuario_id (FK a usuarios)
perfiles_empresa	Gestiona datos corporativos, sector y sitio web de las empresas reclutadoras.	usuario_id (FK a usuarios)
proyectos	Almacena los portfolios y trabajos subidos por los desarrolladores.	autor_id (FK), categoria_id (FK)
ofertas_empleo	Publicaciones de vacantes laborales realizadas por las empresas.	empresa_id (FK a usuarios)

postulaciones	Gestiona la aplicación de desarrolladores a ofertas, vinculando un proyecto como prueba técnica.	oferta_id (FK), desarrollador_id (FK), proyecto_vinculado_id (FK)
evaluaciones	Feedback técnico y notas de administradores sobre los proyectos del portfolio.	proyecto_id (FK), admin_id (FK)
mensajes	Registra la interacción y comunicación por chat entre los distintos usuarios.	emisor_id (FK), receptor_id (FK)
tecnologias	Catálogo de herramientas y lenguajes disponibles para perfiles y ofertas.	categoria_id (FK)
resenas_proyectos	Sistema de comentarios y retroalimentación social para los proyectos.	proyecto_id (FK), usuario_id (FK)
favoritos	Permite a los usuarios guardar perfiles de interés para un seguimiento posterior.	usuario_id (FK), perfil_guardado_id (FK)

5.3.5 PROBLEMAS ENCONTRADOS Y SOLUCIONES

Durante el desarrollo del esquema de datos de la plataforma ProGrammer, se identificaron diversos retos técnicos que requirieron soluciones arquitectónicas específicas para garantizar la integridad, la escalabilidad y el rendimiento del sistema :

5.3.5.1 DIFERENCIACIÓN Y ESPECIALIZACIÓN DE PERFILES

- Descripción del problema: Inicialmente se planteó el uso de una tabla única de usuarios para almacenar toda la información. Sin embargo, se detectó que el diseño resultaba ineficiente, ya que los desarrolladores requerían campos técnicos (como `github_url`) y las empresas requerían datos corporativos (como `sector`), lo que generaba una gran cantidad de valores nulos (sparse data).
- Solución técnica: Se implementó un patrón de herencia por tabla (Table-per-Type). La tabla `usuarios` actúa como entidad base para credenciales comunes, mientras que las tablas `perfiles_desarrollador` y `perfiles_empresa` gestionan los atributos especializados mediante una relación de uno a uno basada en claves foráneas únicas.
- Beneficio: Esta solución asegura la segregación de responsabilidades y optimiza el almacenamiento en disco, permitiendo que el motor de base de datos realice escaneos de tablas más rápidos al reducir el ancho de las filas.

5.3.5.2 GESTIÓN DE SELECCIÓN MÚLTIPLE DE TECNOLOGÍAS

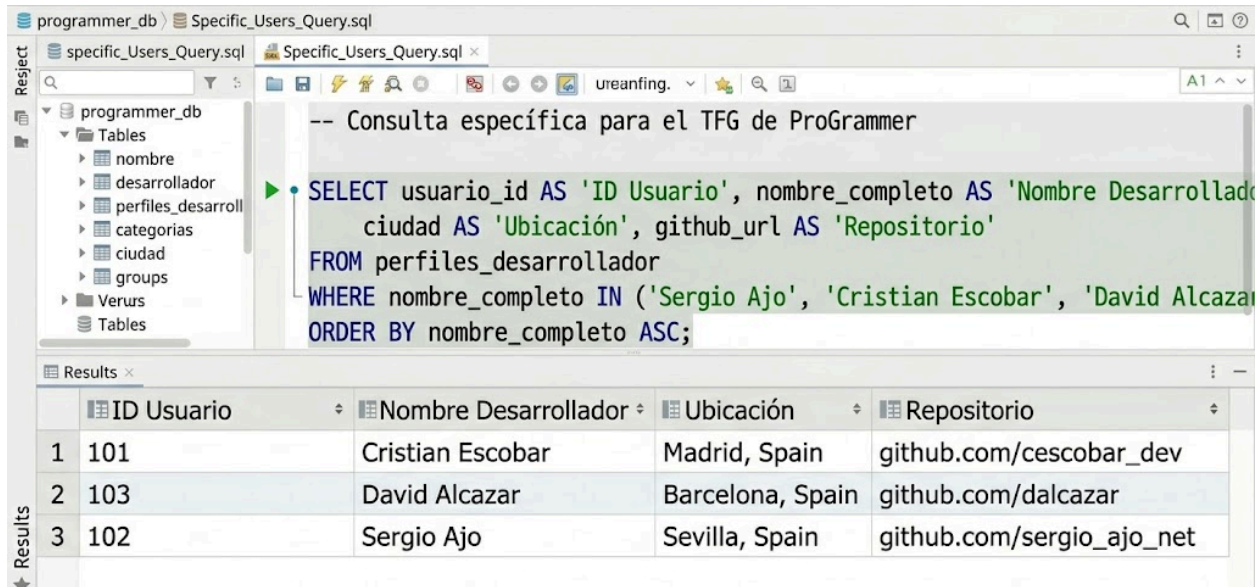
- Descripción del problema: Un reto fundamental fue permitir que un desarrollador o un proyecto estuvieran vinculados a múltiples lenguajes y frameworks simultáneamente. El uso de campos de texto simple habría provocado redundancia de datos e inconsistencias por errores tipográficos.
- Solución técnica: Se diseñó una estructura basada en la Tercera Forma Normal (3NF) mediante una relación de muchos a muchos (N:M). Se creó una tabla maestra de `tecnologias` categorizada y diversas tablas puente (`perfil_tecnologia`, `proyecto_tecnologias`, `oferta_tecnologia`) para gestionar los vínculos.
- Beneficio: El uso de claves primarias compuestas en las tablas de unión garantiza la integridad de los datos, impidiendo duplicidades y facilitando consultas complejas de filtrado que son esenciales para el motor de búsqueda de la plataforma.

5.3.5.3 INTEGRIDAD REF. Y TRAZABILIDAD EN MENSAJERÍA

- Descripción del problema: Se identificó el riesgo de generar registros "huérfanos" al eliminar cuentas de usuario. Si un emisor o receptor desaparecía del sistema, sus mensajes y postulaciones asociadas quedarían sin referencia, provocando errores en la lógica de negocio y en la interfaz de usuario.
- Solución técnica: Se configuraron restricciones de integridad referencial avanzadas mediante claves foráneas duales en la tabla mensajes que apuntan a la entidad usuarios. Se establecieron acciones referenciales como ON DELETE CASCADE para asegurar la limpieza automática de datos dependientes.
- Beneficio: Esta arquitectura autoreferenciada permite una trazabilidad completa de los flujos de comunicación y garantiza que el sistema de notificaciones dinámicas del Header siempre maneje datos coherentes y existentes.

5.3.6 IMPLEMENTACIÓN Y SEGURIDAD

- Tipado de Datos y Hashing: Se han utilizado tipos optimizados como VARCHAR(255) para las contraseñas. Esta longitud es específica para permitir el almacenamiento de hashes generados mediante el algoritmo BCrypt (a través de Spring Security), el cual asegura que la información sensible sea ilegible incluso ante un acceso no autorizado al servidor de datos.
- Gestión de Auditoría Temporal: El uso de TIMESTAMP en campos como fecha_registro permite realizar un seguimiento automático de la actividad sin carga adicional para el backend, facilitando la generación de métricas sobre el crecimiento de la plataforma.
- Restricciones de Unicidad (Constraints): Se han definido restricciones UNIQUE en los campos username y email para garantizar la integridad de la identidad de los usuarios. Esto evita colisiones de datos y errores lógicos en el proceso de autenticación.
- Validación de Calidad Técnica: Mediante la inclusión de restricciones CHECK en las evaluaciones, se obliga a que las notas técnicas se mantengan en un rango de 1 a 10. Esto traslada la validación de la lógica de negocio directamente al motor de la base de datos, actuando como una segunda capa de seguridad ante posibles fallos en la capa de aplicación.



5.4 HERRAMIENTAS Y TECNOLOGÍAS UTILIZADAS

Para el desarrollo del proyecto PROgrammer se han utilizado diversas herramientas, tecnologías y entornos de desarrollo que han permitido llevar a cabo la implementación del sistema de forma eficiente.

En lo que respecta al desarrollo de la interfaz de usuario, se han empleado tecnologías web como HTML, CSS y JavaScript, junto con frameworks y librerías como Bootstrap, jQuery y jQuery UI, que facilitan la creación de interfaces dinámicas y adaptables a distintos dispositivos.

En cuanto al entorno de desarrollo, se ha utilizado Visual Studio Code como herramienta principal, debido a su versatilidad y compatibilidad con las tecnologías empleadas en el proyecto. Asimismo, se ha hecho uso de GitHub como sistema de control de versiones, permitiendo gestionar el desarrollo de forma colaborativa y mantener un control detallado de los cambios realizados en el código.

Para la gestión de la base de datos, se ha utilizado MySQL como sistema gestor, apoyado por herramientas como phpMyAdmin o MySQL Workbench para su administración y modelado.

Por otro lado, se han empleado herramientas complementarias como Google Drive para el almacenamiento de documentación y recursos, Discord para la comunicación entre los

miembros del equipo, Pencil para la elaboración de prototipos de interfaz y Adobe Photoshop para el diseño de elementos gráficos.

En relación con la lógica de negocio del sistema, se ha planteado el uso de tecnologías específicas para el desarrollo backend que permitirán gestionar las operaciones de la aplicación y la comunicación con la base de datos. **[AQUÍ SE HABLARÁ DEL BACKEND Y DE TECNOLOGÍAS COMO SPRING BOOT]**

En conjunto, el diseño del sistema PROgrammer establece las bases técnicas necesarias para su desarrollo, definiendo tanto la estructura de la aplicación como los componentes que la integran y las tecnologías empleadas. Este proceso permite garantizar que el sistema pueda ser implementado de forma organizada, asegurando su coherencia con los requisitos definidos en fases anteriores y facilitando su evolución en etapas posteriores.